CloudNativePG: A Technical and Operational Analysis of a Kubernetes-Native PostgreSQL Operator

August 2026

Abstract

CloudNativePG (CNPG) is a Kubernetes operator for managing PostgreSQL clusters throughout their lifecycle. It uses a declarative Cluster custom resource, Kubernetes reconciliation, PostgreSQL native streaming replication, and an in-container instance manager to provide provisioning, self-healing, failover, backups, and upgrades. This paper examines CNPG's architecture and operating model, focusing on elasticity, failure handling, storage, performance, security, observability, and disaster recovery. It also distinguishes CNPG from StatefulSet-based deployments and from operators that depend on an external high-availability layer. CNPG reduces the number of components involved in PostgreSQL failover, but it does not remove the need for reliable Kubernetes control-plane infrastructure, suitable storage, tested application reconnection behavior, or operational runbooks. CNPG is most suitable for teams that already operate Kubernetes and are prepared to treat database reliability as a property of both the data plane and the control plane.

Keywords: CloudNativePG, PostgreSQL, Kubernetes, database operators, high availability, failover, replication, disaster recovery

1 Introduction

Running PostgreSQL in Kubernetes requires more than placing a database process in a Pod. PostgreSQL has durable state, a primary and standby relationship, write-ahead logging (WAL), replication, client routing, and recovery procedures. Kubernetes supplies resource scheduling, storage abstractions, service discovery, and reconciliation, but it does not by itself understand PostgreSQL's transaction state. An operator connects these two models.

CloudNativePG is a Kubernetes-native PostgreSQL operator designed to manage a highly available primary/standby cluster using PostgreSQL native streaming replication [1, 2]. The desired state is expressed through the CNPG Cluster custom resource (CR). The operator observes that resource and reconciles Pods, PersistentVolumeClaims (PVCs), Services, certificates, and PostgreSQL configuration. The Kubernetes API server therefore acts as the authoritative interface for the cluster's declared topology.

CNPG does not use a StatefulSet to manage PostgreSQL instances. Instead, it directly manages Pods and PVCs. This gives the operator freedom to promote the most suitable standby rather than treating a Pod ordinal as a database identity [2, 3]. Each PostgreSQL Pod contains an instance manager that runs as the parent process and supervises PostgreSQL startup, shutdown, health, and replication-related behavior.

This paper analyzes the model described in CNPG's documentation and the operational consequences of using it. It does not treat claims about Crunchy PostgreSQL for Kubernetes as CNPG features. The distinction is necessary because both projects manage PostgreSQL on Kubernetes but expose different custom resources, commands, images, and lifecycle mechanisms.

2 Methodology and scope

This literature-based technical study is not a benchmark. Its primary sources are the CloudNativePG documentation and project repository, supplemented by Kubernetes and PostgreSQL documentation. It addresses four questions:

1. How does CNPG divide responsibility between Kubernetes, the operator, the instance manager, and PostgreSQL?
2. What operational model does CNPG use for scaling, failover, upgrades, and recovery?
3. Which performance, storage, security, and observability decisions remain the user's responsibility?
4. What trade-offs result from relying on the Kubernetes control plane instead of a separate failover manager?

The supplied source material contains repeated passages and several cross-project references. This paper retains operational concerns that apply to CNPG, but does not attribute other projects' features to it. In particular, the 'PostgresCluster' resource, 'pgo' commands, and Crunchy Data's default pgBackRest configuration are not CNPG behavior.

3 System model

3.1 Control plane and reconciliation

The CNPG operator implements a control loop. A user declares a cluster, including its PostgreSQL image, instance count, storage, replication, backup, and scheduling settings. The operator observes the CR and related Kubernetes resources, compares the desired and actual states, and acts on any difference. Status conditions and events show the result.

This model differs from an imperative runbook in which an administrator executes a fixed sequence of commands and assumes that the result will remain valid. A declarative resource can be reviewed, stored in version control, and reapplied. Reconciliation also supports self-healing after a Pod or node failure. It does not make every change risk free. Storage attachment, scheduling, replication lag, and API availability can delay convergence.

3.2 Main components

The main components are:

- **Cluster custom resource.** This resource describes the intended PostgreSQL cluster and is the principal user-facing configuration object.
- **Operator.** The operator watches the CR and Kubernetes resources, creates and updates workload resources, coordinates lifecycle actions, and publishes cluster status.
- **Instance manager.** A Go process runs as PID 1 in the PostgreSQL container. It manages the PostgreSQL process, probes, shutdown, startup, and instance-level health behavior [3].
- **PostgreSQL.** PostgreSQL provides the database engine, WAL, physical streaming replication, synchronous commit behavior, and recovery mechanisms.
- **Kubernetes resources.** Pods, PVCs, Services, Secrets, ConfigMaps, and certificates provide the execution, storage, routing, and identity primitives used by the cluster.

3.3 Pod and storage management

CNPG creates one Pod per PostgreSQL instance and manages the associated PVCs directly. This avoids the identity and ordering assumptions built into StatefulSets. Kubernetes StatefulSets remain useful for many stateful applications because they provide stable identities and ordered management [10]; CNPG bypasses them because database leadership is determined by PostgreSQL state rather than by an ordinal number.

The direct-management model makes storage policy important. A failed instance may require volume reattachment, a replacement PVC, or recovery from another replica or an object store. The operator can coordinate these steps, but it cannot make an unsuitable storage class durable, fast, or available across failure domains. Production designs should place replicas on separate nodes or availability zones and test the storage behavior expected during node loss.

4 Deployment and configuration

CNPG can be installed with standard Kubernetes tooling, including Helm and Operator Lifecycle Manager distributions where available. After installation, a Cluster CR defines the PostgreSQL workload. A minimal configuration can be concise. Production configurations usually add resource requests, storage, affinity, backup destinations, certificates, monitoring, and disruption policies.

The declarative model supports Infrastructure as Code and GitOps workflows. A configuration change should be made in the CR or in the source repository that generates it, rather than by changing a live Pod or using an unrelated scaling command. This keeps the declared and live instance counts together. If they diverge, the cluster may show confusing status, reconcile an unexpected state, or leave resources behind.

CNPG exposes separate Services for different traffic roles. The read-write service follows the current primary, while read-only services can expose eligible standbys. Applications should use these Services rather than Pod IP addresses. During a promotion, CNPG updates the read-write routing so that clients can locate the new primary [2].

5 Replication, elasticity, and scaling

5.1 Replication model

CNPG uses PostgreSQL physical streaming replication. A cluster normally has one primary and zero or more hot standby instances. Replication may be asynchronous or synchronous. Asynchronous replication permits the primary to continue without waiting for a standby, but a failure can lose transactions that had not reached a surviving replica. Synchronous replication reduces that risk according to the selected quorum and commit settings, at the cost of additional write latency and greater dependence on the availability of synchronous standbys.

The choice is easier to assess in terms of recovery objectives. Recovery point objective (RPO) describes the acceptable amount of data loss, while recovery time objective (RTO) describes the acceptable service interruption. A simple planning relationship is:

$$\text{RPO} \approx \text{unreplicated WAL} + \text{unarchived WAL}. \tag{1}$$

This is not a performance formula. It is a reminder that synchronous replication, asynchronous replication, and WAL archiving protect different failure scopes and should be evaluated together.

5.2 Horizontal scaling

The primary scaling control is the instance count in the Cluster CR. Increasing it causes CNPG to create another PostgreSQL instance and add it to the replication topology. Decreasing it removes an instance according to the operator's lifecycle rules. The edit should be applied through the same configuration path used to manage the cluster. Directly changing a Deployment or issuing an unrelated Kubernetes scale operation is different because CNPG's desired state remains unchanged.

Scaling is not instantaneous. A new instance needs a PVC, a scheduled Pod, an initial copy or recovery process, and sufficient WAL to catch up. Storage provisioning and network bandwidth can dominate the time to readiness. A cluster may report a progressing condition during this period. Administrators should inspect events, PVC status, Pod scheduling, replication state, and operator logs before treating a slow scale operation as a defect.

Scale-out also changes failure behavior. A new replica increases redundancy only after it has caught up and is eligible for the intended service or failover role. A lagging replica may be useful for recovery but may be unsuitable for read traffic or automatic promotion. Scale-in must account for synchronous replica requirements, read traffic, maintenance capacity, and the possibility that a PVC remains for recovery or requires explicit retention policy.

6 Failure handling and failover

6.1 Failover sequence

Failover has two separate outcomes. First, a standby must be promoted to primary. Second, applications must reconnect through a route that identifies the new primary. A successful database promotion without client reconnection is still an application outage.

When the primary becomes unavailable, the operator uses Kubernetes health information and PostgreSQL state to select and promote a suitable standby. The precise behavior depends on the configured failover and switchover settings, the health of the Kubernetes control plane, the availability of replicas, and the amount of unapplied WAL. The read-write Service is then updated to target the promoted instance [3, 2].

CNPG's model does not require Patroni, etcd, ZooKeeper, or Consul as an additional failover manager. This reduces the number of coordination systems that operators must deploy and monitor. It also means that Kubernetes API availability and correctness are part of the database's failover path. A Kubernetes control-plane outage can prevent reconciliation or delay a routing update even when the PostgreSQL processes and storage remain healthy.

6.2 Switchover and testing

A planned switchover is different from an unplanned failover. A switchover can drain or stop the current primary in a controlled manner and should reduce the risk of data loss. An unplanned failover must infer that the primary is unavailable and choose a surviving instance under less favorable conditions.

Organizations should test both paths. A useful drill records failure-detection time, promotion time, Service update time, connection-pool recovery time, and application recovery time. It should also verify a known set of committed writes on the new primary. Tests can include Pod deletion, node drain, node loss, stalled replication, and loss of an availability zone. They should use the same storage, network policy, connection pool, and monitoring configuration as production.

7 Performance and resource management

PostgreSQL performance in Kubernetes depends on the combined behavior of CPU, memory, storage, network, and scheduling. CNPG exposes Pod resource settings, but it cannot infer suitable values from the database schema or workload. Administrators should set requests and limits deliberately and tune PostgreSQL memory parameters within those bounds.

Storage is often the dominant factor. Local volumes may provide predictable latency and high throughput, but they require a recovery design for node loss. Network-attached volumes simplify mobility but can have variable latency and throughput. Before deployment, ‘fio‘ can characterize storage and ‘pgbench‘ can provide a repeatable database workload baseline. The baseline should include latency, throughput, checkpoint behavior, WAL volume, and replica lag rather than only transactions per second.

Separating WAL and data storage can reduce I/O contention where the platform supports it. The benefit depends on the storage devices and workload, so it should be measured rather than assumed. Capacity planning must include data growth, temporary files, WAL retention during replica lag, backups, and the space needed for recovery operations.

Read replicas can serve reporting or read-heavy workloads, but they do not provide arbitrary linear read scaling. Queries routed to a standby observe a replication-defined consistency point and may be delayed. Synchronous replication is primarily a durability decision; using it for every read replica can add write latency without solving application-level consistency requirements.

8 Backups and disaster recovery

Replication is not a backup. A replicated deletion, corruption event, or incorrect transaction can reach every synchronous or asynchronous replica. CNPG supports PostgreSQL backup and recovery workflows using WAL archiving and object storage through its backup mechanisms and plugins. A recovery plan should define where base backups and WAL are stored, how long they are retained, who can restore them, and how restore success is verified [4, 5].

Point-in-time recovery (PITR) provides a way to recover to a time before an operational mistake, provided that the required base backup and WAL are available. Cross-zone or cross-region recovery requires more than an object-store bucket: it requires credentials, network access, compatible images, documented bootstrap configuration, and a tested process for promoting the recovered cluster.

CNPG can support passive or read-only PostgreSQL clusters in other Kubernetes clusters through declarative configuration. The local operator does not provide automatic cross-Kubernetes-cluster failover by itself. A higher-level management system or a documented human procedure must coordinate that decision [2].

9 Security

Security is a property of the complete deployment, not only of the operator image. CNPG supports TLS for client connections and replication, certificate management, custom certificate authorities, and integration with cert-manager. PostgreSQL authentication can also integrate with enterprise mechanisms such as LDAP where configured and supported by the deployment [6, 7].

The Kubernetes configuration should restrict access to the Cluster CR, Secrets, PVCs, and operator resources using least-privilege RBAC. NetworkPolicies should limit database traffic to intended application and replication paths. Backup credentials should be scoped to the required object-store paths, and the object store should enforce encryption, retention, and access logging appropriate to the data.

CNPG writes structured logs to standard output, which allows Kubernetes logging systems to collect database and operator events centrally. Centralized logs should be treated as security-relevant records because they can reveal authentication failures, topology changes, recovery activity, or misconfigured access. The project also documents security practices and publishes its source and release artifacts for review [8].

10 Upgrades and lifecycle management

Operator upgrades and PostgreSQL upgrades are related but distinct. The operator can be upgraded independently of the managed PostgreSQL workload, while a PostgreSQL image change may cause a rolling update, switchover, or require an offline migration depending on the version transition and configuration.

Minor PostgreSQL upgrades can generally use a rolling process because physical replicas can be replaced while another instance remains available. Major upgrades require a different procedure because PostgreSQL does not support in-place data-directory compatibility across major versions. CNPG provides declarative workflows for supported upgrade and import paths, but the application must still be prepared for connection interruptions and the operator must have enough healthy replicas to preserve availability [9].

An upgrade plan should include a tested backup, compatibility checks for extensions, a rollback or restore path, a maintenance communication plan, and post-upgrade validation. Configuration changes that alter Pod templates may trigger a rollout, while other metadata changes may require an explicit restart. Operators should check the relevant version documentation instead of assuming that every manifest change has the same lifecycle effect.

11 Observability and governance

Useful monitoring covers both the PostgreSQL data plane and the Kubernetes control plane. Data-plane indicators include primary and standby role, replication lag, WAL generation, checkpoint behavior, database connections, transaction latency, disk utilization, and backup age. Control-plane indicators include operator health, reconciliation errors, Pod readiness, PVC attachment, scheduling failures, Service endpoints, and certificate expiry.

Alerts should lead to a runbook. For example, a replication-lag alert should identify whether the cause is I/O pressure, network loss, a long-running transaction, insufficient resources, or a backup or recovery operation. A progressing cluster condition should be investigated through events and resource relationships rather than cleared by deleting status or manually removing a resource.

Governance connects technical settings to business requirements. The cluster specification should document the intended RPO, RTO, availability zones, backup retention, maintenance window, owner, and escalation path. Regular recovery exercises provide evidence that the stated objectives are achievable. Without such exercises, a configured backup or failover mechanism is only an assumption.

12 Advantages and limitations

CNPG's principal advantage is a coherent Kubernetes-native control model. Teams do not need to operate a separate failover manager for the local cluster, and the desired topology can be reviewed and reconciled as Kubernetes configuration. Direct Pod and PVC management lets CNPG make leadership and lifecycle decisions based on database state rather than Pod ordinals. Native PostgreSQL replication, Services, backups, certificates, and metrics fit familiar Kubernetes workflows.

The same design creates limits. CNPG depends on Kubernetes API and scheduling behavior for orchestration, so control-plane incidents affect database management and may affect failover completion. The operator cannot compensate for poor storage, a partitioned network, insufficient capacity, or an application that never reconnects. Cross-cluster disaster recovery needs a higher-level authority. Finally, teams moving from Patroni-based or managed PostgreSQL systems must learn new commands, status conditions, runbooks, and assumptions about where high-availability decisions are made.

There is no universal best topology. A single Kubernetes cluster with replicas across zones provides local high availability, while an independent recovery cluster provides a different failure boundary. Synchronous replication protects specific commit paths but can reduce availability when the synchronous member is unavailable. Asynchronous replicas preserve write availability but may lose recent transactions. CNPG makes these choices configurable; it does not eliminate the underlying trade-offs.

13 Discussion

CNPG demonstrates that a database operator can move much of the high-availability control logic into Kubernetes without treating PostgreSQL as a stateless workload. The instance manager handles process-level behavior, PostgreSQL handles replication and recovery, and the operator coordinates resources and topology. This division is simpler than a stack of independent controllers only when the organization already understands Kubernetes and can operate its control plane reliably.

The practical lesson is that declarative configuration does not mean hands-off administration. It shifts administration toward specifying policy, observing reconciliation, and testing the result. Scaling requires storage and replication capacity. Failover requires both promotion and client reconnection. Backups require restoration tests. Security requires Kubernetes, PostgreSQL, certificate, and object-store controls. Upgrades require version-specific validation. These responsibilities remain even when the operator automates the ordinary path.

The model also changes how failure should be investigated. A database incident may begin with PostgreSQL, but the visible symptom may be a Kubernetes Service, a PVC attachment, a node condition, an API timeout, or a connection pool. Effective diagnosis therefore needs a joint view of Kubernetes events, operator status, instance-manager logs, PostgreSQL logs, replication state, and application telemetry.

14 Conclusion

CloudNativePG provides a practical Kubernetes-native model for operating PostgreSQL clusters with declarative configuration, native replication, automated failover, self-healing, backups, and lifecycle automation. Its direct management of Pods and PVCs, together with the in-container instance manager, avoids making StatefulSet ordinals or a separate HA framework responsible for PostgreSQL leadership.

The benefits are strongest when the surrounding platform is well designed. Replicas must be distributed across suitable failure domains, storage must be benchmarked and recoverable, clients must use role-aware Services, backups must be restored in practice, and failover must be rehearsed. CNPG reduces operational complexity, but it does not remove the need for database expertise or control-plane reliability. For Kubernetes teams that accept those responsibilities, CNPG offers a coherent way to place PostgreSQL lifecycle management under a single declarative operating model.

References

[1] CloudNativePG, "CloudNativePG: PostgreSQL Operator for Kubernetes," project website. Available at: https://cloudnative-pg.io/.

[2] CloudNativePG, "Architecture," documentation. Available at: https://cloudnative-pg.io/documentation/current/architecture/.

[3] CloudNativePG, "Postgres instance manager," documentation. Available at: https://cloudnative-pg.io/documentation/current/instance_manager/.

[4] CloudNativePG, "Backup," documentation. Available at: https://cloudnative-pg.io/documentation/current/backup/.

[5] CloudNativePG, "Barman Cloud plugin," documentation. Available at: https://cloudnative-pg.io/documentation/current/recovery/.

[6] CloudNativePG, "Certificates," documentation. Available at: https://cloudnative-pg.io/documentation/current/certificates/.

[7] CloudNativePG, "Authentication," documentation. Available at: https://cloudnative-pg.io/documentation/current/ssl_connections/.

[8] CloudNativePG, "Security," project documentation and repository security material. Available at: https://github.com/cloudnative-pg/cloudnative-pg/security.

[9] CloudNativePG, "Upgrades," documentation. Available at: https://cloudnative-pg.io/documentation/current/upgrade/.

[10] Kubernetes Authors, "StatefulSets," Kubernetes documentation. Available at: https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/.

[11] PostgreSQL Global Development Group, "High Availability, Load Balancing, and Replication," PostgreSQL documentation. Available at: https://www.postgresql.org/docs/current/high-availability.html.

[12] PostgreSQL Global Development Group, "Write-Ahead Logging," PostgreSQL documentation. Available at: https://www.postgresql.org/docs/current/wal.html.

[13] CloudNativePG, "CloudNativePG source repository," GitHub. Available at: https://github.com/cloudnative-pg/cloudnative-pg.